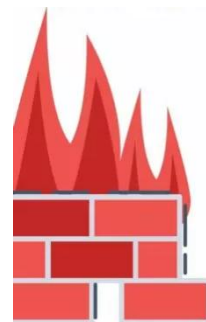


Proteção de perímetro

Introdução

- ✓ Defesa de perímetro refere-se ao conjunto de técnicas e tecnologias que permite assegurar que não existem intrusos dentro da fronteira de rede.
- ✓ A segurança recorre a tecnologias como:
 - Firewalls
 - *Border routers*
- ✓ Evolução da defesa de perímetro
 - Originalmente a defesa era simplesmente realizada com o recurso a um router que permitiam confinar o tráfego a uma só sub-rede (*Border router*).
 - O termo firewall aparece derivado da parede responsável por confinar os incêndios a um só prédio, prevenindo a sua propagação. Aparece pela primeira vez no filme “War Games”



Proteção de perímetro

- ✓ A partir deste ponto a firewall apresenta uma rápida evolução:
 - Firewall por inspeção de pacotes ou *stateless* (1988) – A firewall verifica cada um dos pacotes transmitidos entre pacotes e, baseando-se, numa série de regras rejeita ou aceita os pacotes. As regras mais usuais são o tipo de protocolo e portos de origem e/ou destino
 - Firewall por manutenção de registo ligação ou *statefull* (1989) – A firewall mantém o registo das ligações podendo autorizar ou rejeitar as ligações adicionalmente à existência das regras da firewall *stateless*.
 - Firewall com inspeção de *application layer* (1991) – A firewall tem a capacidade de inspecionar o tráfego e aplicar regras às aplicações utilizadas.
 - Firewall completa, Unified Threat Management (2004) – Firewall com inspeção em *application layer* em que é adicionado anti-virus, anti-spam, etc
 - *Next generation firewall* (2009) - firewall *statefull* em que são adicionadas funcionalidades como, *Application Control*, *URL Filtering*, *Advanced Malware Protection*, *Network Profiling*, *Identity Policy*.

Proteção de perímetro

- *Firewall as a service* (2018) – Firewall NGFW residente na nuvem

Firewall *stateless*

- ✓ A firewall trabalha na filtragem de pacotes, inspecionando o *header* do pacote.
- ✓ O *header* tem a seguinte informação: protocolo, endereço IP de origem, endereço IP de destino, porto. A firewall permite, bloqueia ou ignora um determinado pacote com base em regras que afetam um destes campos.
- ✓ Exemplo:
 - Aceita tráfego para o exterior.
 - `iptables -t raw -I OUTPUT -j NOTRACK`
 - Ignorar os pedidos de ping vindos da subrede 192.168.1.0/24
 - `iptables -A INPUT -p icmp -d 192.168.1.0/24 --icmp-type 8 -j DROP`

Firewall *stateful*

- ✓ A firewall trabalha com regras idênticas à firewall *stateless*.
- ✓ A firewall mantém o registo das ligações. Se a ligação foi permitida, continua a ser permitida.
- ✓ O *header* tem a seguinte informação: protocolo, endereço IP de origem, endereço IP de destino, porto. A firewall permite, bloqueia ou ignora um determinado pacote com base em regras que afetam um destes campos.
- ✓ Exemplo:
 - Aceitar ligações para o exterior já existentes.
 - `iptables -A OUTPUT -m conntrack --ctstate ESTABLISHED -j ACCEPT`
 - aceitar ligações SSH
 - `iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT`

Teste de firewall

Testes simple – SCAN (muitas vezes utilizando nmap)

TCP SYN scan – Aproveitar o facto do RFC793 “obrigar” a uma resposta negativa (com um pacote RST- reset) num pedido de conexão TCP não previsto. Uma firewall corretamente configurada não deve responder a pedidos de SYN.

TCP ACK scan – Aproveitar o facto do RFC793 “obrigar” a uma resposta negativa (com um pacote RST- reset) a pacotes com a flag ACK a “1” sem que existe uma conexão prévia. Uma firewall corretamente configurada não deve responder a pacotes ACK sem que tenha aceitado pedidos de SYN.

UDP scan – Como o protocolo UDP não implica uma conexão, este scan é de resultados mais duvidosos. Normalmente o scan verifica resposta a serviços bem conhecidos em UDP. Ex: DNS

Proteção de perímetro

Testes complexos – PEN (testes em cascata)

1º teste – Verificar se existe firewall, por exemplo utilizando traceroute ao domínio.

2º teste – Verificar que serviços estão acessíveis (TCP SYN, UDP scan)

3º teste – Verificar as versões dos serviços (utilizando o netcat)

```
root@kali:~# nc 192.168.179.146 80
HEAD / HTTP/1.0
HTTP/1.1 400 Bad Request
Date: Tue, 01 Aug 2017 16:26:23 GMT
Server: Apache/2.4.25 (Debian)
Content-Length: 381
Connection: close
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>400 Bad Request</title>
</head><body>
<h1>Bad Request</h1>
<p>Your browser sent a request that this server could not understand.<br />
</p>
<hr>
<address>Apache/2.4.25 (Debian) Server at 127.0.1.1 Port 80</address>
</body></html>
```

4º teste – Verificar a topologia da rede por testes com diferentes TTL (time to live)

Proteção de perímetro

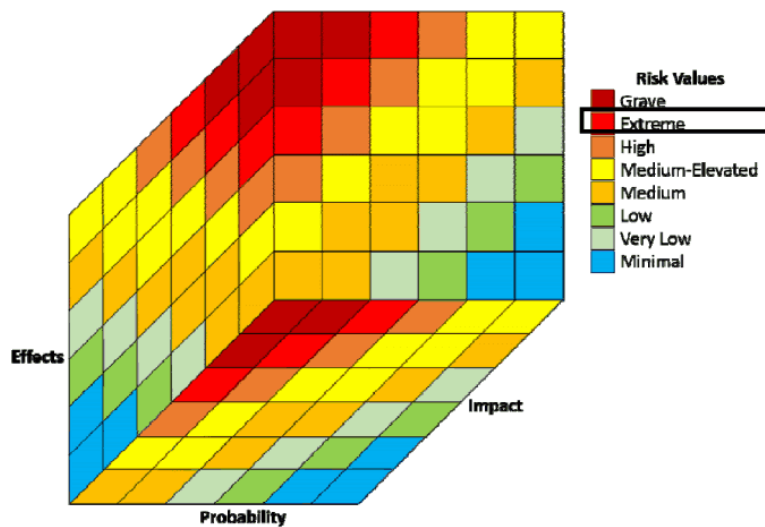
5º teste – Redirecção de portos. Verificar se um porto não está com o valor standard (porto 80 http está no porto 3456). Estes testes são realizados com as ferramentas fpipe e datapipe

6º teste – Verificar a existência de backdoors.

7º teste – http e https tunneling. Verificar se não é possível aceder a outros serviços por HTTP e HTTPS.

8º teste – etc

Proteção de perímetro



<https://purplesec.us/firewall-penetration-testing/>

Proteção de perímetro

Testes com IA (AI-based pen testing)

- ✓ Utiliza MML e agentes autónomos para identificar vulnerabilidades, nomeadamente:
 - Falhas de lógica no modelo de negócio. Scanners tradicionais raramente detetam.
 - Simulação de ataques reais de forma autónoma.
 - Análise em tempo real de padrões e respostas a ataques.
 - Testes aos sistemas de IA com ataques como prompt injection e data poisoning.
 - Priorização de vulnerabilidades

Proteção de perímetro

Tool	Primary Focus	AI/Automation Level	Best For
ZeroThreat.ai	AI-driven web & API pentesting	High (AI-led attack simulation)	Startups, SaaS, DevSecOps teams
Burp Suite	Web app security testing	Medium (scanner + manual)	Security teams & pentesters
Nessus	Vulnerability scanning (network & systems)	High (automated scanning)	IT, security & compliance teams
Metasploit	Exploit development & validation	Medium (framework automation)	Security researchers & red teams
Rapid7	Automated DAST (InsightAppSec)	High (automated scanning)	Enterprises & compliance teams
OWASP ZAP	Open-source DAST	Medium (automated + manual)	Developers & SMBs
Traceable	API security & runtime protection	High (behavioral AI)	API-first companies
Terra Security	Automated web pentesting	High (AI automation)	Growing SaaS companies

<https://secureblitz.com/top-ai-powered-automated-pentesting-tools>

Host Info

Proteção de perímetro

Exemplo de resultado de pentest com IA, host protegido por firewall

Prompt: scan 192.168.3.3

- **Hostname:** shellyplus2pm-a0dd6c28f37c
- **Device:** Shelly Plus 2PM — a smart power relay/energy monitor

Ports Found

Port	State	Service	Notes
53/tcp	Open	DNS	NLnet Labs NSD
80/tcp	Filtered	HTTP	Blocked by firewall/ACL

Notes

- This is an **IoT device** (Shelly Plus 2PM smart switch), which is a common home automation component.
- Port **80 is filtered** — meaning a firewall is dropping packets rather than rejecting them, suggesting there's some network-level protection in front of it, which is good practice for IoT devices.
- Shelly devices run a built-in web UI on port 80 for configuration — keeping it firewalled off while managing it only through Home Assistant or the Shelly Cloud is the right approach.
- Make sure the device's **firmware is up to date** and that it's **not directly exposed to the internet**.

Proteção de perímetro

Prompt: give me more information about this host

Let me run a fuller scan — wider port range, more aggressive service detection, and pull the HTTP content from port 80.

Excellent — the mDNS/Zeroconf service advertisement gave us detailed info. Here's the full picture:

 Full Device Profile — 192.168.3.3

Field	Value
Device	Shelly Plus 2PM
Hostname	ShellyPlus2PM-A0DD6C28F37C
Generation	Gen 2
App	Plus2PM
Firmware	1.7.4
MAC	A0:DD:6C:28:F3:7C
Web UI	Port 80/tcp (filtered from this network)
mDNS/Zeroconf	5353/udp — open and advertising

 Open Services Summary

Port	Proto	State	Service
53/tcp	TCP	Open	DNS (NSD)
80/tcp	TCP	Filtered	HTTP Web UI

Proteção de perímetro

Port	Proto	State	Service
5353/udp	UDP	Open	mDNS/Zeroconf

Security Notes

- **Firmware 1.7.4** — worth checking against the [Shelly changelog](#) to confirm this is the latest Gen 2 release. Shelly has issued security patches in recent versions.
- **Port 80 filtered** — good, the web UI isn't directly accessible from this network segment. Keep it that way.
- **mDNS (5353/udp) is open** — this is how Home Assistant discovers and communicates with it. Normal and expected, but it does advertise the firmware version to anyone on the local network.
- **No MQTT, no CoAP, no UPnP** exposed — clean footprint for an IoT device.

Proteção de perímetro

Exemplo de resultado de pentest com IA, host não protegido por firewall

Prompt: scan 192.168.13.101

Here are the results for **192.168.13.101**:

Host Info

- **Hostname:** home-assistant
- **OS:** Linux 5.x (kernel 5.0–5.14)
- **Status:** Up (2.9ms latency)

Open Ports (1–1000)

Port	Service	Details
22/tcp	SSH	OpenSSH 10.2 (protocol 2.0)
53/tcp	DNS	NLnet Labs NSD
111/tcp	RPCbind	v2–4 (RPC #100000) — also on UDP

Notes

- This is your **Home Assistant** server.

Proteção de perímetro

- **Port 22 (SSH)** — OpenSSH 10.2 is quite recent, good. Make sure password auth is disabled and only key-based login is allowed.
- **Port 111 (RPCbind)** — This is worth noting. RPCbind can expose information about running RPC services and is generally not something you'd want publicly accessible. On a local/VLAN-segmented network it's lower risk, but consider firewalling it if it's not needed.
- **Port 53 (DNS)** — Same NLnet Labs NSD as the UniFi device; likely part of your internal DNS setup.

Home Assistant's typical web UI (port **8123**) wasn't found in the 1–1000 range. Would you like me to scan a wider port range or check specifically for port 8123?

Prompt: test 8123

Port 8123 is open and confirmed as **Home Assistant**:

Home Assistant Web UI — Port 8123

Detail	Value
Port	8123/tcp — open
Service	HTTP — aiohttp 3.13.3
Runtime	Python 3.14
Title	Home Assistant
robots.txt	Disallows /

Notes

Proteção de perímetro

- Running on **aiohttp 3.13.3** / **Python 3.14** — very recent versions, good sign.
- The UI is served over **plain HTTP** (not HTTPS). If this is accessible outside your local network or VLAN, that's a risk — credentials and tokens would be transmitted unencrypted. Consider putting it behind a reverse proxy with TLS (e.g., your nginx on the UniFi device, or a dedicated proxy).
- `robots.txt` blocking / is just cosmetic — it doesn't provide any security.

Would you like me to run a Nikto scan against port 8123 as well?

Prompt: yes

Nikto is unable to connect to port 8123, even though Nmap confirmed it's open. This is likely because:

- **Home Assistant actively blocks or drops connections** from unknown/automated scanners — it has built-in brute-force and abuse protection that can reject rapid or unusual HTTP requests.
- **Nikto's version is outdated** (flagged in every run), which may affect its ability to handle aiohttp/modern HTTP stacks correctly.
- There may be a **firewall rule or rate limiter** on the HA instance blocking the scanner's source IP after the Nmap probe.

What you can do instead

- **Manual review** — browse to `http://192.168.13.101:8123` and check the HA Security settings under *Settings* → *System* → *Security*
- Key things to verify in Home Assistant:
 - **HTTPS enabled** (via the `homeassistant` or a reverse proxy config)
 - **Trusted networks** properly scoped
 - **Multi-factor authentication** enabled for your user accounts
 - **Exposed to the internet?** — if so, ensure it's behind Nabu Casa or a properly configured reverse proxy with TLS

Proteção de perímetro

Would you like me to try a Gobuster directory scan against it instead, or move on to scanning another host?

Prompt: yes

Gobuster is up and running. Here are the results:

Gobuster Directory Scan — `http://192.168.13.101:8123`

Path	Status	Notes
/app	200	App section
/config	200	⚠ Configuration panel
/home	200	Dashboard
/history	200	Entity history/logs
/map	200	Device tracker map
/profile	200	User profile
/security	200	⚠ Security settings

Notes

- All paths return **200 OK** with identical size (5948 bytes) — this is typical of Home Assistant's **single-page app (SPA)** design, where the frontend router handles all paths and the server returns the same `index.html` shell regardless of route.
- `/config` and `/security` are the most sensitive routes; make sure MFA is enforced.

The biggest remaining concern is still the **lack of HTTPS** on port 8123. Want me to scan any other hosts on your network?

Regras de firewall

✓ Classificação:

- Regras de acesso: Regras que permitem, bloqueiam ou rejeitam acessos a portos, protocolos ou endereços (*stateless*)
- Regras de filtragem *stateful*: permite ou não a ligação.
- Regras de NAT: Regras que permitem “transformar” um IP noutra.
- Regras de alteração de características de tráfego: Permitem banda de ligação de acordo com o regras de censor (ex: QOS, *traffic shapping*). Estas regras não são “tudo ou nada”.

Regras de acesso e filtragem

✓ *Checklist* para construção de regras (SANS Institute):

- Regras *anti-spoofing*: Não permite redes privadas vindas do exterior

Proteção de perímetro

- Regras de utilizador. Permitir ligações a serviços
 - Regras de gestão. Permitir ligações a protocolos de serviço (SNMP, OSPF)
 - Regras de ruído. Limitar protocolos como OSPF fora do âmbito das redes desejadas.
 - Regras de alerta: Alertar o gestor de determinada tentativa de ligação.
 - Regras de log: Colocar a tentativa de ligação num log
- ✓ Cada zona (interface =>rede) tem regras específicas e/ou partilhadas. Por norma, cada zona, quando criada, bloqueia todo o tráfego
 - ✓ As regras de NAT são verificadas antes (quando na LAN existe um servidor) ou depois das regras de firewall (quando na LAN existe um cliente), ou seja, é necessário muito cuidado com a sequência.
 - ✓ As regras de firewall são verificadas sequencialmente de cima para baixo. As regras são aplicadas por ordem de grupo:

Proteção de perímetro

1. Regras de sistema. Por exemplo, regras que garantem que não ficamos sem acesso à firewall.
 2. Regras flutuantes: Regras que são independentes da zona da firewall.
 3. Grupos de interfaces. Conjunto de interfaces que foram agrupadas.
 4. Regras de interface. Regras só aplicadas a uma interface
- ✓ As regras, por norma, são *stateful*. Desta forma, se uma conexão se mantiver a regra não é verificada, novamente.
- Se a regra for *stateless*, todos os pacotes são confrontados com as regras
 - Se a regra for *stateless*, não é verificada a ordem do pacote e o *timestamp* dos mesmos => podemos ter um problema de IP *spoofing* (o que é isto??)
 - IP *spoofing*: Alteração do endereço de IP da fonte no *header* do pacote
 - IP *spoofing* utilizado em ataques DDoS (o que é isto ??) em que o IP da fonte é o da vítima.

Proteção de perímetro

- Ataques DDoS pretendem “deitar abaixo” a rede da vítima não permitindo a sua normal operação. Isto é realizado com milhares de clientes
- ✓ As regras têm ações:
 - *Pass*: permite o tráfego.
 - *Block*: Não permite o tráfego e não avisa o cliente que foi bloqueado.
 - *Reject*: Não permite o tráfego e avisa o cliente. Muito pouco utilizado pois não queremos que saibam da nossa existência.
- ✓ As regras IPV4 e IPV6 por norma são diferentes (muitas vezes duplicadas) mas podem ser combinadas (uma regra para ambas versões).
- ✓ As regras têm uma rede/endereço de destino.
 - Não confundir rede WAN com a internet, a rede WAN é simplesmente a interface com o IP público da firewall. Permitir tráfego para a interface WAN não permite o tráfego para a internet.

Proteção de perímetro

- ✓ As regras têm uma rede/endereço de fonte. Usualmente a rede para a qual estamos a criar a regra.
- ✓ As regras podem ser de entrada (IN) da fonte para a firewall ou saída (OUT) da firewall para o destino. Ex:
 - Permitir o acesso da rede 10.10.10.0/24 à rede 10.10.11.0/24. Regra IN na rede 10.1.10.0/24 com destino a rede 10.1.11.0/24



- ✓ Inversão de destino: Regras aplicadas a todas as redes não listadas no destino. Ex:
 - Permitir o acesso à internet: Lista de todas as redes internas como destino, mas invertidas => todas as outras redes do mundo
- ✓ Regras no defender do Windows?: Muito mais simples, regras *inbound* e *outbound* por tipo de rede: *Public*/Domain/*Private*

Proteção de perímetro

- ✓ Regras no UFW Linux?: Ainda mais simples, regras *inbound* e *outbound*

Regras de NAT

- ✓ NAT permite transformar endereços separando a rede interna (IP's privados) da rede externa (IP público)
- ✓ Como na firewall existe a fonte e o destino. No entanto, existe igualmente o *redirect* (para onde o tráfego deve ser dirigido), este facto aparece quando a relação é de m:n se for 1:n não existe.
- ✓ Port forwarding ou Destination NAT (DNAT). Ligação iniciada na interface WAN, permite mapear um porto da rede WAN num IP/porto de uma das redes internas
 - Regra aplicada a portos na interface WAN
 - A rede de destino é uma das redes internas
 - O IP de destino pertence à rede de destino

Proteção de perímetro

- Os portos de destino pertencem ao IP de destino
- ✓ *Outbound* ou *source* NAT (SNAT). Regra inversa ao DNAT. Ligação iniciada no IP privado. Tipicamente criadas automaticamente pela firewall.
- ✓ Regras de acesso podem estar misturadas com regras de *port forwarding*.
 - Muitas da firewall (Ex: router dos ISP) funciona com a não separação completa entre regras NAT e regras de acesso.
 - Facilita a vida ao comum dos utilizadores.
 - Jogar *League of legends* => realizar *forward* dos portos TCP: 2099, 5222-5223, 8088, 8393-8400 UDP: 5000-5500, 8088.
 - Como o comum dos utilizadores acha que dá muito trabalho criar regras, apareceu a *Universal Plug and Play*. As Aplicações podem criar regras NAT por própria autoria. Nunca se deve utilizar, mas:
 - 1º link no google: “mais correto é não habilitar o UPnP”.

Proteção de perímetro

- 2º link do google: “Quer seja em uma casa inteligente ou em um escritório, dispositivos IoT funcionam melhor quando estão interconectados com outros aparelhos próximos. O UPnP une todos esses dispositivos sem nenhum contratempo.”. Fabuloso, certo?

Regras de *traffic shapping*

- ✓ Cada regra está associada a uma *pipe*. A *pipe* é uma descrição da ligação à internet: Largura de banda, atraso de propagação (*ping*), perda de pacotes, etc.
- ✓ Cada tipo de tráfego está associado a um *Flow*: Um flow é caracterizado pela porta e tipo de protocolo, ex: ligação HTTP: porto 80, TCP
- ✓ Dentro de cada *pipe* podemos determinar o peso relativo de cada *flow* criando uma *Queue*, garantindo prioridade ao tráfego “mais interessante

Proteção de perímetro

Regras auxiliares

- ✓ *Regras flutuantes*: Regras independentes da subrede. Liberta o utilizador de ter de criar as mesmas regras para cada zona.
- ✓ *Aliases*: Liberta o utilizador de ter de recorrer a mnemónicas ou ao papel porque permite criar nomes comuns para interfaces, redes, servidores etc.

Regras de zona

- ✓ Regras aplicadas a conjunto de vlan's (zona) e não somente a uma subrede, facilita uma visão do conjunto e criação de regras por nível de segurança.

Proteção de perímetro

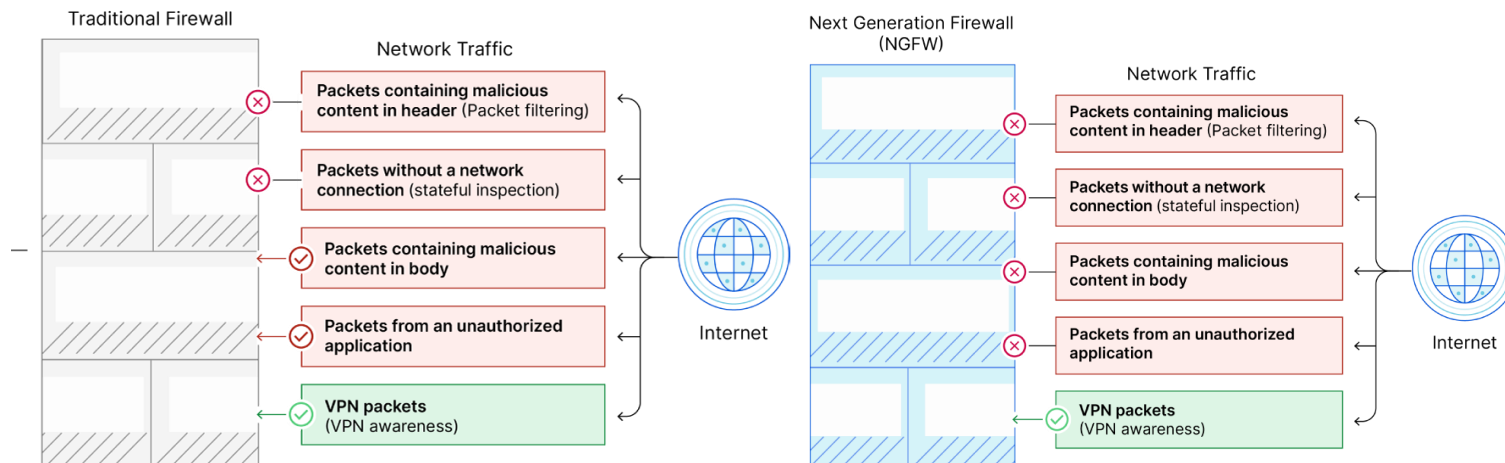
Built-in Zones		Destination Zone					
		Internal	External	Gateway	VPN	Hotspot	DMZ
Source Zone	Internal	Allow All	Policies	Allow All	Allow All	Allow All	Allow All
	External	Policies	Policies	Policies	Policies	Policies	Policies
	Gateway	Allow All	Allow All	-	Allow All	Allow All	Allow All
	VPN	Allow All	Policies	Allow All	Allow All	Allow All	Allow All
	Hotspot	Allow Return Traffic	Policies	Policies	Allow Return Traffic	Block All	Block All
	DMZ	Allow Return Traffic	Policies	Policies	Allow Return Traffic	Block All	Block All

<https://help.ui.com/hc/en-us/articles/115003173168-Zone-Based-Firewalls-in-UniFi>

Proteção de perímetro

Next Generation Firewall

- ✓ Não existe uma definição única das capacidades de uma NGFW.
 - Gartner: Adiciona capacidades de inspeção do comportamento das aplicações e prevenção de intrusão recorrendo muitas vezes a base de dados externas.;



<https://www.cloudflare.com/learning/security/what-is-next-generation-firewall-ngfw/>

Proteção de perímetro

- ✓ Nem todas as capacidades estão disponíveis em todos os produtos
- ✓ São o maior consumidor de CPU e memória numa firewall
- ✓ Muitas das capacidades podem ser implementadas, separadamente, por outros produtos. Por exemplo: DPI com o suricata, ou análise de tráfego com NAGIOS.
 - Todas as capacidades a funcionar dentro de uma firewall pode ser inviável
 - Produtos especializados têm, normalmente, um desempenho superior.
- ✓ As capacidades são aplicadas com o recurso a políticas. Por exemplo políticas para crianças ou para adultos.

Capacidades

- ✓ Controlo de aplicações: Verificação do tipo de aplicação por inspeção do pacote de dados ou destino dos mesmos, possibilidade de bloquear grupos de aplicações como por exemplo: Redes sociais. Possibilidade de bloquear ao nível da aplicação: Por exemplo Facebook

Proteção de perímetro

- ✓ Controlo de aplicações residentes em *cloud*: O mesmo tipo de verificação, mas a aplicação está residente num servidor e não na máquina do utilizador.
- ✓ Análise de tráfego: Verificação do tipo e qualidade do tráfego recorrendo cada vez mais a IA com reconhecimento de padrões
- ✓ Controlo de acesso a sites: Verificação e controlo de acesso a grupos de sites (com campanhas de *phishing*, sites recentes, redes sociais etc) ou mesmo a sites individuais;
- ✓ Controlo de acesso a sites “perigosos” recorrendo a bases de dados externas classificadores (*Cloud Threat Intelligence*), por exemplo, a Bright tem 4,4 mil milhões de URL’s classificados;
 - Controlo de acesso a sites com comportamento estranhos. O site realiza redireccionamentos não usuais, existe JavaScript considerado malicioso etc.
 - Controlo de acesso por URL
- ✓ Controlo de dispositivos ligados à rede com aplicação de políticas por grupos ou individuais

Proteção de perímetro

- ✓ *Traffic shaping* e QOS aplicação de banda diferenciada consoante o tipo de tráfego
- ✓ Prevenção de ataques com pacotes encriptados. Por exemplo por prevenção na utilização de VPN
- ✓ *Deep packet inspection* com tráfego não encriptado ou encriptado (alguns casos como TLS)